

抖音增长客户端方向 - 简版消息列表实现技术文档

1. 整体架构设计

本项目基于 **Android Studio** 集成开发环境，采用 **Java** 语言进行原生开发。架构设计上严格遵循 **MVVM (Model-View-ViewModel)** 模式，结合 **Android Jetpack** 组件库，实现了 UI 视图、业务逻辑与数据存储的深度解耦，确保了代码的可维护性与扩展性。

1.1 架构分层说明

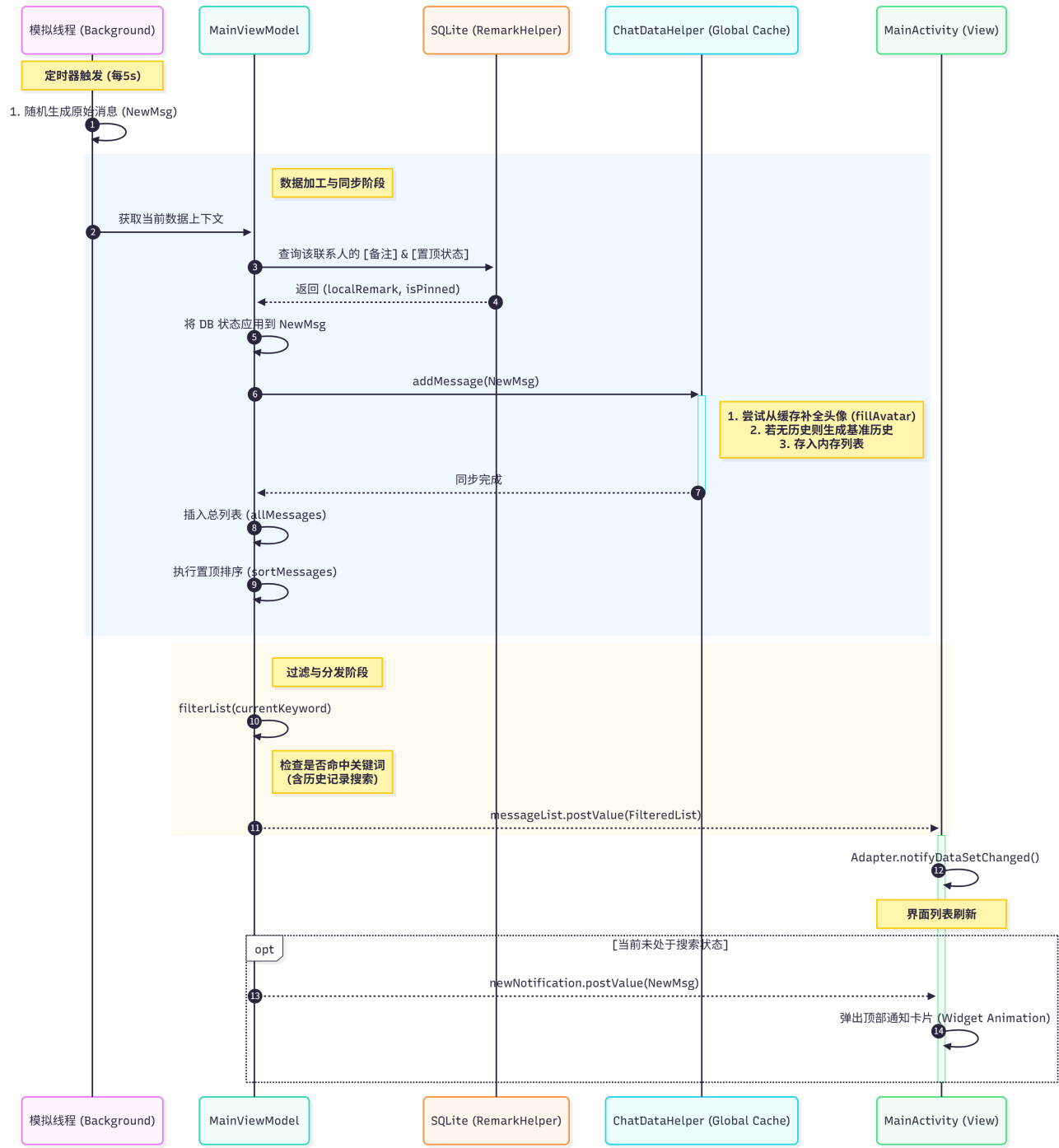
- **View 层 (视图交互层):**
 - **技术栈:** `Activity`, `XML Layouts`, `ViewBinding`, `RecyclerView`。
 - **职责:** 负责 UI 渲染与用户交互响应。
 - **特性:**
 - **类型安全:** 全面采用 `ViewBinding` 技术替代传统的 `findViewById`，在编译时确保空安全与类型安全，杜绝空指针异常。
 - **复杂列表渲染:** 使用 `RecyclerView` 配合自定义 `Adapter` (`MessageAdapter`, `ChatAdapter`)，通过 `getItemViewType` 实现多 `ViewType` 的高效复用与渲染。
 - **被动更新:** View 层不持有数据状态，而是通过观察 (`Observe`) `ViewModel` 暴露的 `LiveData` 来响应界面变化。
- **ViewModel 层 (业务逻辑与状态管理层):**
 - **技术栈:** `ViewModel`, `LiveData`, `Thread`。
 - **职责:** 作为 UI 的状态持有者 (`StateHolder`)，负责处理业务逻辑与数据转换。
 - **特性:**
 - **生命周期感知:** 利用 `ViewModel` 存活于配置变更（如屏幕旋转）期间的特性，防止数据丢失。
 - **数据驱动:** 通过 `MutableLiveData` 分发数据，实现单向数据流。
 - **逻辑中枢:** 托管后台模拟线程，处理搜索过滤算法 (`filterList`)、消息置顶排序 (`Comparator`) 及未读数逻辑，从 UI 控制器中剥离复杂逻辑。
- **Model 层 (数据仓库层):**
 - **技术栈:** `SQLiteOpenHelper`, `Gson`, `Java Collections`。
 - **职责:** 负责数据的获取、缓存与持久化，为上层提供单一信源。
 - **分级存储策略:**
 1. **静态数据源:** 启动时通过 `Gson` 解析 `assets` 目录下的 JSON 文件，模拟冷启动数据。
 2. **内存缓存:** 构建 `ChatDataHelper` 单例仓库，维护全局消息队列与头像缓存，保障多页面间的数据强一致性。
 3. **持久化存储:** 基于原生 `SQLite` (`RemarkDatabaseHelper`) 实现用户配置（备注、电话、置顶状态）的本地落地，并支持 Schema 版本迁移。

类图



消息流转时序图

展示后台模拟产生新消息并更新 UI 的流程：



2. 功能点实现方式

2.1 消息列表页

- **实现方式:**
 - 使用 `RecyclerView` 配合 `MessageAdapter`。
 - 通过重写 `getItemViewType` 方法，支持多种 Item 类型：
 - `TYPE_ITEM`: 普通消息。
 - `TYPE_FOOTER`: 底部加载状态条
 - **UI 细节:** 实现了头像、昵称、时间（使用 `TimeUtils` 格式化）、未读红点、置顶。

2.2 数据源与分页

- **数据源:** 启动时使用 `Gson` 解析 `assets/data.json`。
- **上滑加载更多 & 模拟分页:**
 - 监听 `RecyclerView` 的滑动状态。
 - 首次进入页面展示20 条消息。`loadMoreData` 逻辑中，上滑到底部显示加载圈，加载更多数据。当模拟数据耗尽时，底部显示“暂时没有更多了”。

2.3 备注页与持久化

- **交互:** 点击头像跳转备注页面 -> `RemarkActivity`。
- **功能:** 展示头像、原昵称，提供“备注名”和“电话”的编辑框。
- **存储:**
 - 使用 `SQLiteOpenHelper`。
 - 表结构设计为 (`nickname` PRIMARY KEY, `remark`, `is_pinned`, `phone`)。
 - 使用 `INSERT OR REPLACE` 或 `UPDATE` 逻辑确保数据更新。

2.4 消息内容的类型化展示

- **列表页 (List View):**
 - 根据消息类型 (`TYPE_TEXT`, `TYPE_IMAGE`, `TYPE_CARD`, `TYPE_SYSTEM_TEXT_CARD`)，在列表摘要处显示差异化的文本（如“[图片]”、“分享商品：xxx”），提升信息识别效率。
- **聊天页 (Chat View):**
 - `ChatAdapter` 通过 `getItemViewType` 动态加载左右两侧（我/对方）的气泡布局。
 - **普通文本:** 支持长文本自动换行与背景自适应。
 - **图片消息:** 使用 `Glide` 加载网络或本地图片，并进行圆角裁剪处理。
 - **运营卡片:**
 - 用于展示商品、地点等富媒体信息。
 - **UI 结构:** 顶部大图封面 + 底部标题与副标题（价格/距离）的组合布局，支持点击交互。
 - **系统通知卡片:**
 - 用于展示系统级的重要通知。
 - **UI 结构:** 无图纯文本设计，采用白色圆角卡片风格，底部包含分割线及“确认/详情”双操作按钮。

2.5 消息时间文案规则

- **核心类**：通用的工具类 `TimeUtils`，将 Unix 时间戳转换为符合人类阅读习惯的字符串。
- **算法逻辑**：
 1. 获取当前系统时间戳 `now` 和消息时间戳 `timeMillis`。
 2. 计算时间差 `span = now - timeMillis`。
 3. **规则判定**（按优先级）：
 - `span < 1分钟` → 返回 "刚刚"。
 - `span < 1小时` → 计算 `span / 60000`，返回 "`xx`分钟前"。
 - `timeMillis > 今天0点` → 使用 `SimpleDateFormat("HH:mm")`，返回 "`12:30`"。
 - `timeMillis > 昨天0点` → 返回 "昨天 `HH:mm`"。
 - `timeMillis > 7天前` → 计算天数差，返回 "`x`天前"。
 - 其他情况 → 返回 "`MM-dd`" (如 "11-05")。

2.6 搜索消息

- **逻辑**：
 - 封装了通用的 `filterList(keyword)` 方法。
 - 支持搜索：昵称、备注、卡片标题、聊天记录内容。
 - 当搜索命中历史记录时，动态构建一个“替身消息对象”，将其 `content` 替换为命中的那句历史信息，使得命中消息能够在消息列表高亮显示。
- **UI**：
 - 使用 `SpannableString` 和 `ForegroundColorSpan` 实现关键词红色高亮。

2.7 自定义转场动画

- **进场**：使用 Android 5.0+ 的 **Shared Element Transition**。
 - 配置 `ChangeBounds` 和 `ChangeTransform`。
 - 使用 `postponeEnterTransition` 等待 RecyclerView 渲染完成后再开始动画，防止闪烁。
- **退场 (跟手下滑)**：
 - 在 `ChatActivity` 中自定义 `OnTouchListener` 监听标题栏。
 - 根据手指移动距离动态改变 View 的 `TranslationY` 和 `Alpha`。
 - 松手时判断距离，触发回弹或 `finish` 动画。

2.8 沉浸式 UI 与键盘适配

- **沉浸式**：使用 `android:fitsSystemWindows="true"`。
- **键盘适配**：
 - Manifest 配置 `adjustResize`。
 - 使用 `addOnLayoutChangeListener` 监听列表高度变化，当键盘弹出时自动将 RecyclerView 滚动到底部，确保最新消息可见。

2.9 轻量触达模拟

- **UI 布局**：
 - 在 `MainActivity` 的根布局最上层叠加了一个 `CardView` 模拟 Android 顶部浮窗通知。
 - 包含头像、昵称、消息摘要和装饰性图标，默认状态为 `View.GONE`。
- **触发机制**：
 - `MainViewModel` 中定义了 `MutableLiveData<Message> newNotification`。
 - 当后台线程生成新消息且当前**未处于搜索状态**时，向 `newNotification` 发送数据。

- **动画交互:**
 - **进场:** 使用 `ViewPropertyAnimator` 实现 `TranslationY` 从负值（屏幕外）滑入 0（屏幕内），配合 `FastOutSlowInInterpolator` 插值器模拟物理弹跳效果。
 - **驻留与消失:** 使用 `Handler.postDelayed(runnable, 3000)` 实现 3 秒自动收起。
 - **点击跳转:** 点击 `CardView` 直接跳转 `ChatActivity` 并传递消息实体，同时调用 `clearUnread` 清除红点。

2.10 消息置顶功能

- **数据层:**
 - `Message` 实体类增加 `boolean isPinned` 字段。
 - `RemarkDatabaseHelper` 升级至 Version 2, `user_remarks` 表新增 `is_pinned` 列 (INTEGER)。
- **排序算法 (Comparator):**
 - 在 `MainViewModel` 中实现了自定义 `Comparator<Message>`。
 - **排序规则:**
 1. **优先级一:** 置顶状态 (`o1.isPinned` vs `o2.isPinned`)。置顶者排在非置顶者之前。
 2. **优先级二:** 若置顶状态相同，则保持原有的时间顺序（新消息在列表头部，旧消息在后）。
- **交互逻辑:**
 - `MessageAdapter` 监听 Item 长按事件，弹出“置顶/取消置顶”对话框。
 - 用户操作后，更新内存对象状态 -> 写入 SQLite 数据库 -> 触发 `Collections.sort()` -> 刷新 UI。
- **状态保持:**
 - 当后台模拟器生成新消息时，代码会先检查旧列表中该联系人的 `isPinned` 状态，并强制赋值给新消息对象，随后立即调用 `sortMessages()`，确保置顶的联系人即使收到新消息，也依然置顶在列表最上方。

2.12 弱网/无网体验

为了提升极端网络环境下的用户体验，项目引入了 **多状态视图管理机制**，确保在数据加载过程中给予用户明确的反馈。

- **状态定义:** 在 `ViewModel` 中定义了三种页面状态常量：
 - `STATE_LOADING`: 数据请求中，展示骨架屏。
 - `STATE_ERROR`: 请求失败（如超时/无网），展示错误页与重试按钮。
 - `STATE_CONTENT`: 请求成功，展示正常的消息列表。
- **实现机制:**
 1. **骨架屏 (Skeleton UI):**
 - 使用 `FrameLayout` 覆盖在列表上方，内部包含模拟列表项的灰色色块布局 (`item_skeleton.xml`)。
 - 配合显示的加载圈 `ProgressBar`，提供更加生动的加载中视觉反馈。
 2. **故障模拟与超时:**
 - 在 `loadData()` 中使用 `Thread.sleep(5000)` 模拟 5 秒的网络超时延迟。
 - 设置 `isFirstLoad` 标志位，强制首次加载进入“失败状态”，展示错误页面，用于演示异常处理流程。
 3. **状态驱动 UI:**
 - `MainActivity` 观察 `pageState LiveData`。
 - 当状态变更时，通过 `setVisibility` 互斥地控制 **骨架屏容器**、**错误页容器** 和 **SwipeRefreshLayout** 的显示与隐藏。

4. 重试机制：

- 错误页包含一个“点击重试”按钮。
- 点击后重新调用 `viewModel.loadData()`，重置状态为 `STATE_LOADING`，再次发起数据请求（此时模拟成功逻辑）。

3. 数据库设计与迁移方案

3.1 数据库 Schema

数据库名称: `douyin_lite.db` 表名: `user_remarks`

字段名	类型	约束	说明	版本引入
<code>nickname</code>	TEXT	PRIMARY KEY	用户昵称（作为唯一标识）	V1
<code>remark</code>	TEXT		用户设置的备注名	V1
<code>is_pinned</code>	INTEGER	DEFAULT 0	置顶状态 (1=置顶, 0=普通)	V2
<code>phone</code>	TEXT		用户备注的电话号码	V3

3.2 迁移方案

在 `SQLiteOpenHelper.onUpgrade(db, oldVersion, newVersion)` 中实现平滑迁移，确保老用户数据不丢失。

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    // 升级到 V2: 增加置顶字段
    if (oldVersion < 2) {
        db.execSQL("ALTER TABLE user_remarks ADD COLUMN is_pinned INTEGER DEFAULT 0");
    }
    // 升级到 V3: 增加电话字段
    if (oldVersion < 3) {
        db.execSQL("ALTER TABLE user_remarks ADD COLUMN phone TEXT");
    }
}
```

4. 消息中心 / 动态插入机制说明

为了模拟高频的消息收发场景，项目构建了一套基于后台线程的**消息分发中心 (Message Dispatcher)**，实现了从消息生成、状态继承到全链路同步的完整闭环。

4.1 模拟线程与生命周期

在 `MainViewModel` 初始化时启动一个守护线程 (`Daemon Thread`)，模拟长连接的消息推送机制。

- **生成频率：**通过 `Thread.sleep(5000)` 设置 5 秒的时间间隔，模拟消息到达。

- **消息内容随机策略：**

- **联系人随机：**从预设的 `mockSenders` 数组中随机抽取发信人。
- **内容体裁随机：**按概率分布生成不同类型的消息体（60% 文本、20% 图片、20% 运营卡片），并自动匹配对应的系统通知类型。

4.2 状态继承机制 (State Inheritance)

这是本模块的核心逻辑。新消息不仅仅是一个全新的对象，它必须继承该联系人已有的“历史状态”，以保证用户体验的连续性。

1. **置顶状态继承：**生成新消息时，先查询 `allMessages` 列表或 `RemarkDatabaseHelper`。如果该联系人已被用户置顶 (`isPinned=true`)，新消息强制继承此属性，确保其插入列表后依然停留在顶部区域，而不是被错误地按时间排序沉底。
2. **备注名继承：**查询 SQLite 数据库，将用户设置的本地备注 (`Local Remark`) 应用到新消息对象上，防止新消息到来时昵称突然变回原名。
3. **头像补全：**利用 `ChatDataHelper` 的全局头像缓存，自动回填 `avatarUrl`，解决后台模拟生成的消息对象因缺乏上下文而导致头像丢失的问题。

4.3 数据一致性与全链路同步 (Data Consistency)

为了解决“列表页摘要”（View Model 层）与“聊天详情页历史聊天记录”（Activity 层）数据源割裂的问题，引入了 `ChatDataHelper` 作为内存中的单一信源。

- **同步写入策略：**

- 当 `MainViewModel` 生成一条新消息后，在更新 UI 列表之前，必须先调用 `ChatDataHelper.addMessage()`。
- 该方法会将新消息追加到全局内存仓库的对应联系人列表中，确保用户随后点击进入聊天页时，能看到这条最新的消息记录。

- **首次触达即生成：**

- **触发时机：**无论是“收到新消息”还是“用户主动点击对话框”，只要系统检测到内存中没有该联系人的历史记录，立即调用 `generateBaseHistory`，随机生成 5-10 条带有时间梯度的历史聊天记录作为“数据基底”。
- **结果：**所有新产生的消息都会追加到这个基底之后，解决了只有新消息，没有旧历史记录的数据断层问题。

4.4 UI 响应机制

- **列表刷新：**通过 `LiveData (messageList)` 将处理好的新列表投递给主线程，、`notifyDataSetChanged` 触发 `RecyclerView` 刷新。
- **轻量触达：**如果当前不在搜索模式下，通过 `newNotification` `LiveData` 触发 `MainActivity` 展示顶部的 **In-App Notification Widget**，模拟系统级通知弹窗。

5. 遇到的问题与解决方案

问题 1: `ConcurrentModificationException` 崩溃

- **现象：**当后台线程正在插入新消息，而主线程正在遍历列表（如刷新备注或搜索）时，APP 崩溃。
- **原因：**Java 集合在遍历时被另一个线程修改结构。

- **解决方案:**
 - 在 `MainViewModel` 的过滤和处理逻辑中, 使用 `try-catch` 捕获异常。
 - 在进行搜索过滤时, 创建原列表的 **副本 (Snapshot)** `new ArrayList<>(allMessages)` 进行遍历操作。

问题 2: 聊天页头像显示异常/丢失

- **现象:** 模拟生成的新消息或历史消息在聊天页显示为默认头像。
- **原因:** 后台模拟的新消息对象只包含了基础文本, 缺失了头像 URL/ID 信息; 且历史生成逻辑未同步头像。
- **解决方案:**
 - 建立全局 `avatarCache` (Map)。
 - 在 APP 启动加载 JSON 时, 预存所有人的头像信息。
 - 在生成新消息或历史消息时, 强制调用 `fillAvatar()` 方法从缓存中回填头像数据。

问题 3: 搜索结果被新消息“冲刷”

- **现象:** 用户正在搜索关键词, 结果列表显示正常。几秒后后台生成一条无关新消息, 界面刷新, 搜索结果消失, 变成了全部消息列表。
- **原因:** 后台线程更新 UI 时, 未检查当前是否处于搜索状态, 直接投递了完整列表。
- **解决方案:**
 - 封装统一的 `filterList(list, keyword)` 方法。
 - 后台线程在 `postValue` 之前, 检查 `currentKeyword`。如果有关键词, 则使用 `filterList` 对新生成的完整列表再次过滤, 确保 UI 始终显示符合当前搜索条件的数据。

问题 4: 软键盘遮挡底部输入框

- **现象:** 点击输入框, 软键盘弹出盖住了输入栏, 无法看到输入内容。
- **原因:** 全屏/沉浸式布局导致 `adjustResize` 失效。
- **解决方案:**
 - Manifest 中设置 `android:windowSoftInputMode="adjustResize"`。
 - 布局根节点添加 `android:fitsSystemWindows="true"`, 让系统重新接管 Padding 计算。
 - 添加 `OnLayoutChangeListener`, 检测到列表高度变小 (键盘弹出) 时, 自动将 RecyclerView 滚动至底部 (`scrollToPosition`)。